02157 Functional Programming
Week 5:
Collections: Finite Sets and Maps

Sebastian Mödersheim
Based on slides by Michael Reichhardt Hansen

October 2, 2026

02157 Functional Programming
Week 5:
Collections: Finite Sets and Maps

Sebastian Mödersheim
Based on slides by Michael Reichhardt Hansen

October 2, 2026

Overview

- Conventions and tradition
  – example with higher-order list functions

Overview

- Conventions and tradition
  – example with higher-order list functions

- Sets and Maps as abstract data types
  - ★ Useful when modelling and when programming
  - ★ Many similarities with the list library

Conventions and tradition

exemplified using the Flights and Luggage problem
from Week 4
– an old exam question

Conventions and tradition

- type names are descriptive and start with capital letters

exemplified using the Flights and Luggage problem
from Week 4
– an old exam question

Conventions and tradition

- type names are descriptive and start with capital letters
- function names are descriptive and start with small letters

exemplified using the Flights and Luggage problem
from Week 4
– an old exam question

Conventions and tradition

- type names are descriptive and start with capital letters
- function names are descriptive and start with small letters
- variable names are short and consistently used

exemplified using the Flights and Luggage problem
from Week 4
– an old exam question

Conventions and tradition

- type names are descriptive and start with capital letters
- function names are descriptive and start with small letters
- variable names are short and consistently used
- function types are stated in comments

exemplified using the Flights and Luggage problem
from Week 4
– an old exam question

Conventions and tradition

- type names are descriptive and start with capital letters
- function names are descriptive and start with small letters
- variable names are short and consistently used
- function types are stated in comments
- types are avoided in function declarations – unless needed

exemplified using the Flights and Luggage problem
from Week 4
– an old exam question

Conventions and tradition

- type names are descriptive and start with capital letters
- function names are descriptive and start with small letters
- variable names are short and consistently used
- function types are stated in comments
- types are avoided in function declarations – unless needed
- self-check that wanted type is an instance of inferred type
- ....

exemplified using the Flights and Luggage problem
from Week 4
– an old exam question

Example: Flights and Luggage (1)

- Type names are descriptive and
- start with capital letters

```
type Lid     = string
type Flight  = string
type Airport = string

type Route   = (Flight * Airport) list

type LuggageCatalogue = (Lid * Route) list

let lc1 =
   [("DL 016-914",
       [("DL 189","ALT");("DL 124","BRU");("SN 733","CPH")]);
    ("SK 222-142",
       [("SK 208","ALT");("DL 124","BRU");("SK 122","JFK")])]
```

Example: Flights and Luggage (2)

- function names are descriptive and start with small letters
- variable names are short and consistently used
- function types are stated in comments
- types are avoided in function declarations – unless needed
- self-check that wanted type is an instance of inferred type

```
// Lid * LuggageCatalogue -> Route
let rec findRoute lid lc =
   match lc with
   | [] -> None
   | ((lid',route)::_) when lid'=lid -> Some route
   | (_::lc') -> findRoute lid lc'
```

Example: Flights and Luggage (2)

- function names are descriptive and start with small letters
- variable names are short and consistently used
- function types are stated in comments
- types are avoided in function declarations – unless needed
- self-check that wanted type is an instance of inferred type

```
// Lid * LuggageCatalogue -> Route
let rec findRoute lid lc =
   match lc with
   | [] -> None
   | ((lid',route)::_) when lid'=lid -> Some route
   | (_::lc') -> findRoute lid lc'
```

Notice that further comments are not needed and that

- lid, lid', lid1 .... denote luggage identifiers
- f, f', f1, ... denote flights
- lc, lc', lc1, lcrest denote luggage catalogues
- r, r', r1, .. denote routes

Example: Flights and Luggage (3)

```
// inRoute: Flight -> Route -> bool
let rec inRoute f =
   function
   | (f1,_)::r -> f=f1 || inRoute f r
   | []        -> false
```

Example: Flights and Luggage (3)

```
// inRoute: Flight -> Route -> bool
let rec inRoute f =
   function
   | (f1,_)::r -> f=f1 || inRoute f r
   | []        -> false
```

`List.exists` is the natural choice
– the existence of an element with a certain property is questioned

```
let inRoute f r = List.exists (fun (f1,_) -> f=f1) r
```

Example: Flights and Luggage (4)

- the function `inRoute` is handy to use

```
// withFlight: Flight -> LuggageCatalogue -> Lid list
let rec withFlight f =
   function
   | []                              -> []
   | (lid,r)::lc when inRoute f r -> lid::withFlight f lc
   | _::lc                           -> withFlight f lc
```

Example: Flights and Luggage (4)

- the function `inRoute` is handy to use

```
// withFlight: Flight -> LuggageCatalogue -> Lid list
let rec withFlight f =
    function
    | []                                -> []
    | (lid,r)::lc when inRoute f r -> lid::withFlight f lc
    | _::lc                             -> withFlight f lc
```

- One can see that as a filtering: all those elements of the flight catalog where the given flight is `inRoute`.
- That gives a list of pairs (l, r) and we just want the first of each pair:

```
let withFlight f lc =
  let lrs = List.filter (fun (lid,route) -> inRoute f route) lc
  List.map fst lrs
```

or very briefly:

```
let withFlight f =
  List.map fst << (List.filter (fun (lid,route) -> inRoute f route))
```

Example: Flights and Luggage (5)

```
type ArrivalCatalogue = (Airport * Lid list) list
```

The task:

$$\text{extend} : \text{Lid} * \text{Route} \to \text{ArrivalCatalogue} \to \text{ArrivalCatalogue}$$

Example: Flights and Luggage (5)

```
type ArrivalCatalogue = (Airport * Lid list) list
```

The task:

$$\text{extend} : \text{Lid} * \text{Route} \to \text{ArrivalCatalogue} \to \text{ArrivalCatalogue}$$

Invent a function to make things easy

```
// insert: Lid -> Airport
                -> ArrivalCatalogue->ArrivalCatalogue
let rec insert lid airp =
   function
   | []                -> [(airp, [lid])]
   | (airp1,ls)::rest when airp1=airp
                       -> (airp1,lid::ls)::rest
   | pair::rest        -> pair::insert lid airp rest
```

Example: Flights and Luggage (5)

```
type ArrivalCatalogue = (Airport * Lid list) list
```

The task:

$$\text{extend} : \text{Lid} * \text{Route} \to \text{ArrivalCatalogue} \to \text{ArrivalCatalogue}$$

Invent a function to make things easy

```
// insert: Lid -> Airport
                -> ArrivalCatalogue->ArrivalCatalogue
let rec insert lid airp =
   function
   | []                -> [(airp, [lid])]
   | (airp1,ls)::rest when airp1=airp
                       -> (airp1,lid::ls)::rest
   | pair::rest        -> pair::insert lid airp rest


let extend (lid,route) ac =
  List.fold (fun ac (fl,ap) -> insert lid ap ac) ac route
```

Example: Flights and Luggage (6)

```
type LuggageCatalogue = (Lid * Route) list
type ArrivalCatalogue = (Airport * Lid list) list
```

The task:

$$\text{toArrivalCatalogue} : \text{LuggageCatalogue} \to \text{ArrivalCatalogue}$$

should be solved using a fold function and

$$\text{extend} : \text{Lid} * \text{Route} \to \text{ArrivalCatalogue} \to \text{ArrivalCatalogue}$$

Example: Flights and Luggage (6)

```
type LuggageCatalogue = (Lid * Route) list
type ArrivalCatalogue = (Airport * Lid list) list
```

The task:

$$\text{toArrivalCatalogue} : \text{LuggageCatalogue} \to \text{ArrivalCatalogue}$$

should be solved using a fold function and

$$\text{extend} : \text{Lid} * \text{Route} \to \text{ArrivalCatalogue} \to \text{ArrivalCatalogue}$$

```
let toArrivalCatalogue ac = List.foldBack extend ac []
```

Example: Flights and Luggage (6)

```
type LuggageCatalogue = (Lid * Route) list
type ArrivalCatalogue = (Airport * Lid list) list
```

The task:

$$\text{toArrivalCatalogue} : \text{LuggageCatalogue} \rightarrow \text{ArrivalCatalogue}$$

should be solved using a fold function and

$$\text{extend} : \text{Lid} * \text{Route} \rightarrow \text{ArrivalCatalogue} \rightarrow \text{ArrivalCatalogue}$$

```
let toArrivalCatalogue ac = List.foldBack extend ac []
```

Naming conventions and tradition support readability

Example: Flights and Luggage (6)

```
type LuggageCatalogue = (Lid * Route) list
type ArrivalCatalogue = (Airport * Lid list) list
```

The task:

$$\text{toArrivalCatalogue} : \text{LuggageCatalogue} \rightarrow \text{ArrivalCatalogue}$$

should be solved using a fold function and

$$\text{extend} : \text{Lid} * \text{Route} \rightarrow \text{ArrivalCatalogue} \rightarrow \text{ArrivalCatalogue}$$

```
let toArrivalCatalogue ac = List.foldBack extend ac []
```

Naming conventions and tradition support readability

– and so does functional decomposition

FSharp's immutable collections

Sets and Maps

FSharp's immutable collections

- List: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is significant
    - ▶ $[1; 2; 3] \neq [3; 2; 1]$
  - ★ repetitions among elements of a list are significant
    - ▶ $[1; 1; 3] \neq [1; 3]$

FSharp's immutable collections

- List: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is significant
    - ▶ $[1; 2; 3] \neq [3; 2; 1]$
  - ★ repetitions among elements of a list are significant
    - ▶ $[1; 1; 3] \neq [1; 3]$

- Set: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is not significant
    - ▶ $\{1, 1, 3\} = \{1, 3, 1\}$
  - ★ repetitions among elements of a set are not significant
    - ▶ $\{1, 1, 3\} = \{1, 3\}$

Today

- Multi-set: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is not significant
    - ▶ $\langle 1, 2, 3\rangle = \langle 3, 2, 1\rangle$
  - ★ repetitions among elements of a multi-set are significant
    - ▶ $\langle 1, 1, 3\rangle \neq \langle 1, 3\rangle$

Today's exercises

FSharp's immutable collections

- List: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is significant
    - ▶ $[1; 2; 3] \neq [3; 2; 1]$
  - ★ repetitions among elements of a list are significant
    - ▶ $[1; 1; 3] \neq [1; 3]$

- Set: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is not significant
    - ▶ $\{1, 1, 3\} = \{1, 3, 1\}$
  - ★ repetitions among elements of a set are not significant
    - ▶ $\{1, 1, 3\} = \{1, 3\}$

Today

- Multi-set: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is not significant
    - ▶ $\langle 1, 2, 3\rangle = \langle 3, 2, 1\rangle$
  - ★ repetitions among elements of a multi-set are significant
    - ▶ $\langle 1, 1, 3\rangle \neq \langle 1, 3\rangle$

Today's exercises

- Map: a partial function from a domain of keys to values
  - ★ only defined for finitely many keys

Today

FSharp's immutable collections

- List: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is significant
    - ▶ $[1; 2; 3] \neq [3; 2; 1]$
  - ★ repetitions among elements of a list are significant
    - ▶ $[1; 1; 3] \neq [1; 3]$

- Set: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is not significant
    - ▶ $\{1, 1, 3\} = \{1, 3, 1\}$
  - ★ repetitions among elements of a set are not significant
    - ▶ $\{1, 1, 3\} = \{1, 3\}$

Today

- Multi-set: a finite collection of elements of the same type
  - ★ the order in which elements are enumerated is not significant
    - ▶ $\langle 1, 2, 3\rangle = \langle 3, 2, 1\rangle$
  - ★ repetitions among elements of a multi-set are significant
    - ▶ $\langle 1, 1, 3\rangle \neq \langle 1, 3\rangle$

Today's exercises

- Map: a partial function from a domain of keys to values
  - ★ only defined for finitely many keys

Today

- Sequence: a possibly infinite list of elements of the same type
  - ★ the elements of a sequence are computed by demand

Covered later in the semester

Types, data types and abstract data types

- A type is generated from basic types `int`, `float`, `bool`, `string`, `...` and type variables `'a`, `'b`, `'c`, `...` using type operators `*`, `->`, `list`, `...`

Types, data types and abstract data types

- A type is generated from basic types `int, float, bool, string, ...` and type variables `'a, 'b, 'c, ...` using type operators `*, ->, list, ...`

- A data type is characterized by
  - ★ a type — `'a list`
  - ★ a set of values — `[ ]`, `[v]`, `[v1; ...vn]`
  - ★ a set of operations — `::`, `@` , `List.rev`, `List.fold`, ...

Types, data types and abstract data types

- A type is generated from basic types `int, float, bool, string, ...` and type variables `'a, 'b, 'c, ...` using type operators `*, ->, list, ...`

- A data type is characterized by
  - ★ a type — `'a list`
  - ★ a set of values — `[ ]`, `[v]`, `[v1; ...vn]`
  - ★ a set of operations — `::`, `@` , `List.rev`, `List.fold`, ...

- A abstract data type is a data type
  - ★ where the representation of values is hidden — Liskov & Zilles 1974

Types, data types and abstract data types

- A type is generated from basic types `int, float, bool, string, ...` and type variables `'a, 'b, 'c, ...` using type operators `*, ->, list, ...`

- A data type is characterized by
  - ★ a type — `'a list`
  - ★ a set of values — `[ ]`, `[v]`, `[v1; ...vn]`
  - ★ a set of operations — `::`, `@` , `List.rev`, `List.fold`, ...

- A abstract data type is a data type
  - ★ where the representation of values is hidden — Liskov & Zilles 1974

Examples:

- `List` is a data type but not an abstract one
  — the representation of list values is visible (`[]` and `::`)

- `Set` and `Map` are abstract data types

The set concept (1)

A *set* (in mathematics) is a collection of element like

$$\{\text{Barbara}, \text{Bob}, \text{Ben}\}, \{1, 3, 5, 7, 9\}, \mathbb{N}, \text{and } \mathbb{R}$$

- the order in which elements are enumerated is not significant
- repetitions among elements of a set are not significant

The set concept (1)

A *set* (in mathematics) is a collection of element like

$$\{\text{Barbara}, \text{Bob}, \text{Ben}\}, \{1, 3, 5, 7, 9\}, \mathbb{N}, \text{and } \mathbb{R}$$

- the order in which elements are enumerated is not significant
- repetitions among elements of a set are not significant

It is possible to decide whether a given value is in the set.

$$\text{Alice} \notin \{\text{Barbara}, \text{Bob}, \text{Ben}\} \qquad \text{and} \qquad 7 \in \{1, 3, 5, 7, 9\}$$

The set concept (1)

A *set* (in mathematics) is a collection of element like

$$\{\text{Barbara}, \text{Bob}, \text{Ben}\}, \{1, 3, 5, 7, 9\}, \mathbb{N}, \text{and } \mathbb{R}$$

- the order in which elements are enumerated is not significant
- repetitions among elements of a set are not significant

It is possible to decide whether a given value is in the set.

$$\text{Alice} \notin \{\text{Barbara}, \text{Bob}, \text{Ben}\} \qquad \text{and} \qquad 7 \in \{1, 3, 5, 7, 9\}$$

The empty set containing no element is written $\{\}$ or $\emptyset$.

The sets concept (2)

A set A is a *subset* of a set B, written $A \subseteq B$, if all the elements of A are also elements of B, for example

$$\{\text{Ben}, \text{Bob}\} \subseteq \{\text{Barbara}, \text{Bob}, \text{Ben}\} \qquad \text{and} \qquad \{1, 3, 5, 7, 9\} \subseteq \mathbb{N}$$

The sets concept (2)

A set A is a *subset* of a set B, written $A \subseteq B$, if all the elements of A are also elements of B, for example

$$\{\text{Ben}, \text{Bob}\} \subseteq \{\text{Barbara}, \text{Bob}, \text{Ben}\} \qquad \text{and} \qquad \{1, 3, 5, 7, 9\} \subseteq \mathbb{N}$$

Two sets A and B are equal, if they are both subsets of each other:

$$A = B \qquad \text{if and only if} \qquad A \subseteq B \text{ and } B \subseteq A$$

i.e. two sets are equal if they contain exactly the same elements.

The sets concept (2)

A set A is a *subset* of a set B, written $A \subseteq B$, if all the elements of A are also elements of B, for example

$$\{\text{Ben}, \text{Bob}\} \subseteq \{\text{Barbara}, \text{Bob}, \text{Ben}\} \qquad \text{and} \qquad \{1, 3, 5, 7, 9\} \subseteq \mathbb{N}$$

Two sets A and B are equal, if they are both subsets of each other:

$$A = B \qquad \text{if and only if} \qquad A \subseteq B \text{ and } B \subseteq A$$

i.e. two sets are equal if they contain exactly the same elements.

The subset of a set A which consists of those elements satisfying a predicate p can be expressed using a *set-comprehension*:

$$\{x \in A \mid p(x)\}$$

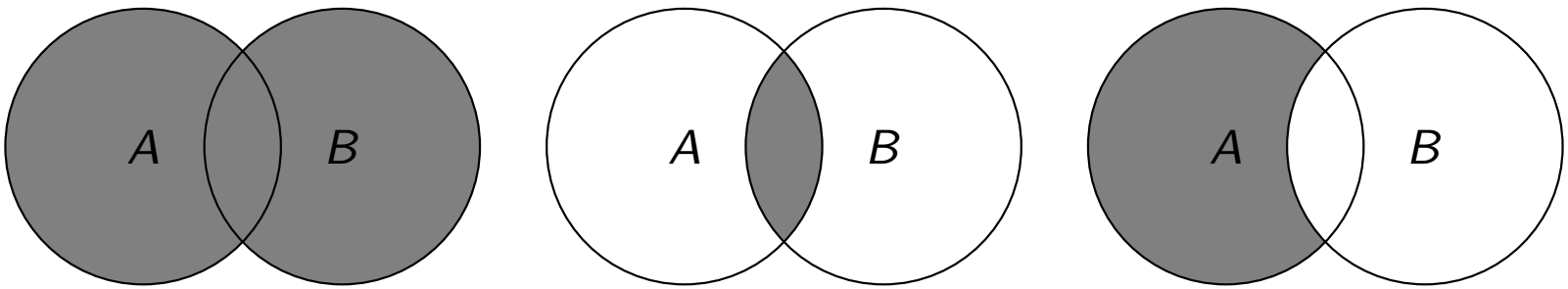
For example:

$$\{1, 3, 5, 7, 9\} = \{x \in \mathbb{N} \mid \text{odd}(x) \text{ and } x < 11\}$$

The set concept (3)

Some standard operations on sets:

$$
\begin{array}{lcll}
A \cup B & = & \{x \mid x \in A \text{ or } x \in B\} & \text{union} \\
A \cap B & = & \{x \mid x \in A \text{ and } x \in B\} & \text{intersection} \\
A \setminus B & = & \{x \in A \mid x \notin B\} & \text{difference}
\end{array}
$$



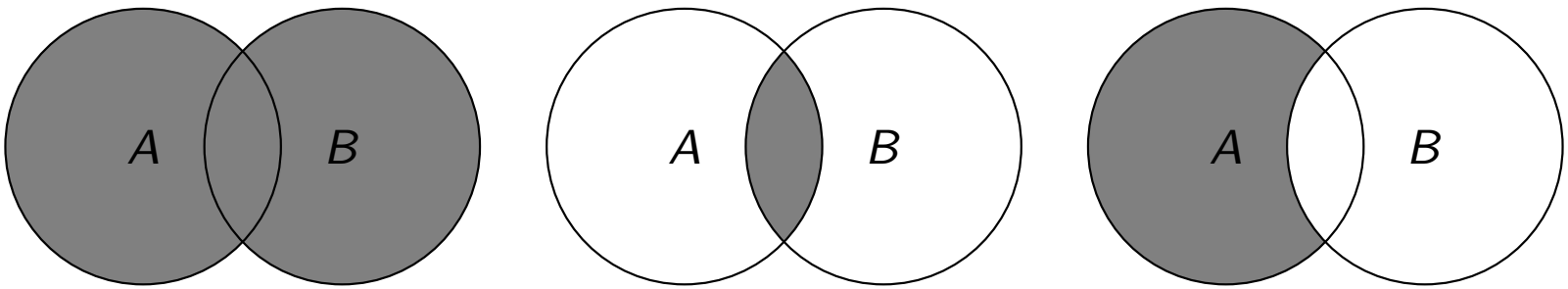
(a) $A \cup B$ (b) $A \cap B$ (c) $A \setminus B$

Figure: Venn diagrams for (a) union, (b) intersection and (c) difference

The set concept (3)

Some standard operations on sets:

$$
\begin{array}{lcll}
A \cup B & = & \{x \mid x \in A \text{ or } x \in B\} & \text{union} \\
A \cap B & = & \{x \mid x \in A \text{ and } x \in B\} & \text{intersection} \\
A \setminus B & = & \{x \in A \mid x \notin B\} & \text{difference}
\end{array}
$$



(a) $A \cup B$ (b) $A \cap B$ (c) $A \setminus B$

Figure: Venn diagrams for (a) union, (b) intersection and (c) difference

For example

$$
\begin{array}{lcl}
\{\text{Barbara}, \text{Bob}, \text{Ben}\} \cup \{\text{Alice}, \text{Barbara}, \text{Ann}\} & = & \{\text{Alice}, \text{Ann}, \text{Barbara}, \text{Bob}, \text{Ben}\} \\
\{\text{Barbara}, \text{Bob}, \text{Ben}\} \cap \{\text{Alice}, \text{Barbara}, \text{Ann}\} & = & \{\text{Barbara}\} \\
\{\text{Barbara}, \text{Bob}, \text{Ben}\} \setminus \{\text{Alice}, \text{Barbara}, \text{Ann}\} & = & \{\text{Bob}, \text{Ben}\}
\end{array}
$$

Abstract Data Types

An Abstract Data Type: A type together with a collection of operations, where

- the representation of values is hidden.

Abstract Data Types

An Abstract Data Type: A type together with a collection of operations, where

- the representation of values is hidden.

An abstract data type for sets must have:

- Operations to generate sets from the elements. Why?
- Operations to extract the elements of a set. Why?
- Standard operations on sets.

Immutable Sets in F#

An Abstract Data Type: `Set<'a>`

An abstract type for sets should at least support the following:

```
empty:      Set<'a>
add:        'a -> Set<'a> -> Set<'a>
union:      Set<'a> -> Set<'a> -> Set<'a>
intersect:  Set<'a> -> Set<'a> -> Set<'a>
difference: Set<'a> -> Set<'a> -> Set<'a>
contains:   'a -> Set<'a> -> bool
toList:     Set<'a> -> 'a list
```

where

- any finite set can be generated by repeatedly adding elements to the empty set;
- union, intersection and difference are fundamental set operations;
- `contains` and `toList` are used to inspect the set

An Abstract Data Type: `Set<'a>`

An abstract type for sets should at least support the following:

```
empty:      Set<'a>
add:        'a -> Set<'a> -> Set<'a>
union:      Set<'a> -> Set<'a> -> Set<'a>
intersect:  Set<'a> -> Set<'a> -> Set<'a>
difference: Set<'a> -> Set<'a> -> Set<'a>
contains:   'a -> Set<'a> -> bool
toList:     Set<'a> -> 'a list
```

where

- any finite set can be generated by repeatedly adding elements to the empty set;
- union, intersection and difference are fundamental set operations;
- `contains` and `toList` are used to inspect the set

Note:

- the above operations are supported by the library `Set`.
- the representation of sets used by `Set` is hidden from the user.

Finite sets in F#

The Set library of F# supports finite sets. An efficient implementation is based on balanced binary trees.

Examples:

```
set ["Bob"; "Barbara"; "Ben"];;
val it : Set<string> = set ["Ben"; "Barbara"; "Bob"]

set [3; 1; 9; 5; 7; 9; 1];;
val it : Set<int> = set [1; 3; 5; 7; 9]
```

Equality of two sets is tested in the usual manner:

```
set["Bob";"Barbara";"Ben"] = set["Barbara";"Ben";"Barbara";"Bob"];;
val it : bool = true
```

Sets are ordered on the basis of a lexicographical ordering:

```
compare (set ["Ann";"Jane"]) (set ["Barbara";"Ben";"Bob"]);;
val it : int = -1
```

Immutability of `Set<'a>`

```
let s = Set.ofList [3; 2; 0];;
val s : Set<int> = set [0; 2; 3]

Set.add 1 s;;
val it : Set<int> = set [0; 1; 2; 3]

s;;
val it : Set<int> = set [0; 2; 3]
```

Note: Do not read this wrong:

`Set.add` is not a function that changes a set, but that returns a set.
Evaluation of `Set.add 1 s` does not change `s` but returns the set $\{1\} \cup s$.

Selected further operations (1)

- `ofList: 'a list -> Set<'a>`,
  where `ofList` $[a_0; \ldots; a_{n-1}] = \{a_0; \ldots; a_{n-1}\}$
- `remove: 'a -> Set<'a> -> Set<'a>`,
  where `remove` $a\ A = A \setminus \{a\}$
- `minElement: Set<'a> -> 'a`
  where `minElement` $\{a_0, a_1, \ldots, a_{n-2}, a_{n-1}\} = a_0$ when $n > 0$
  (assuming that the enumeration respect the ordering)

Selected further operations (1)

- `ofList: 'a list -> Set<'a>`,
  where `ofList` $[a_0; \ldots; a_{n-1}] = \{a_0; \ldots; a_{n-1}\}$
- `remove: 'a -> Set<'a> -> Set<'a>`,
  where `remove` $a\ A = A \setminus \{a\}$
- `minElement: Set<'a> -> 'a`
  where `minElement` $\{a_0, a_1, \ldots, a_{n-2}, a_{n-1}\} = a_0$ when $n > 0$
  (assuming that the enumeration respect the ordering)

Notice that minElement on a non-empty set is well-defined due to the ordering:

```
Set.minElement (Set.ofList ["Bob"; "Barbara"; "Ben"]);;
val it : string = "Barbara"
```

Selected further operations (2)

- `filter: ('a -> bool) -> Set<'a> -> Set<'a>`, where `filter` $p\ A = \{x \in A \mid p(x)\}$
- `exists: ('a -> bool) -> Set<'a> -> bool`, where `exists` $p\ A = \exists x \in A.p(x)$
- `forall: ('a -> bool) -> Set<'a> -> bool`, where `forall` $p\ A = \forall x \in A.p(x)$
- `fold: ('a -> 'b -> 'a) -> 'a -> Set<'b> -> 'a`, where

$$\begin{aligned}&\texttt{fold}\ f\ a\ \{b_0, b_1, \ldots, b_{n-2}, b_{n-1}\}\\ &= f(f(f(\cdots(f(f\ a\ b_0)\ b_1)\cdots)\ b_{n-2})\ b_{n-1})\end{aligned}$$

Selected further operations (2)

- `filter: ('a -> bool) -> Set<'a> -> Set<'a>`, where `filter` $p\ A = \{x \in A \mid p(x)\}$
- `exists: ('a -> bool) -> Set<'a> -> bool`, where `exists` $p\ A = \exists x \in A.p(x)$
- `forall: ('a -> bool) -> Set<'a> -> bool`, where `forall` $p\ A = \forall x \in A.p(x)$
- `fold: ('a -> 'b -> 'a) -> 'a -> Set<'b> -> 'a`, where

$$\begin{aligned}&\texttt{fold}\ f\ a\ \{b_0, b_1, \ldots, b_{n-2}, b_{n-1}\}\\ &= f(f(f(\cdots(f(f\ a\ b_0)\ b_1)\cdots)\ b_{n-2})\ b_{n-1})\end{aligned}$$

These work similar to their List siblings, e.g.

$$\texttt{Set.fold (-) 0 (set [1; 2; 3])} = ((0-1)-2)-3 = -6$$

where the ordering is exploited.

Example: Airport Luggage System

```
type ArrivalCatalogue = (Airport * Lid list) list

type ArrivalCatalogue = (Airport,Lid Set) Set
```

Outline

1. Immutable Maps in F#

The map concept

A *map* from a set A to a set B is a *finite* subset A' of A together with a *function* m defined on A': $m : A' \to B$.
The set A' is called the *domain* of m: $\mathrm{dom}\, m = A'$.

A map m can be described in a tabular form:

a_0	b_0
a_1	b_1
⋮	
a_{n-1}	b_{n-1}

- An element a_i in the set A' is called a *key*
- A pair (a_i, b_i) is called an *entry*, and
- b_i is called the *value* for the key a_i.

We denote the sets of entries of a map as follows:

$$\mathrm{entriesOf}(m) = \{(a_0, b_0), \ldots, (a_{n-1}, b_{n-1})\}$$

Keys are unique

Selected map operations in F#

- `ofList: ('a*'b) list -> Map<'a,'b>`
  `ofList` $[(a_0, b_0); \ldots; (a_{n-1}, b_{n-1})] = m$
- `add: 'a -> 'b -> Map<'a,'b> -> Map<'a,'b>`
  `add` a b $m = m'$, where m' is obtained m by overriding m with the entry (a, b)
- `find: 'a -> Map<'a,'b> -> 'b`
  `find` a $m = m(a)$, if $a \in \mathrm{dom}\ m$;
  otherwise an exception is raised
- `tryFind: 'a -> Map<'a,'b> -> 'b option`
  `tryFind` a $m =$ `Some` $(m(a))$, if $a \in \mathrm{dom}\ m$; `None` otherwise
- `foldBack:('a->'b->'c->'c) -> Map<'a,'b> -> 'c -> 'c`
  `foldBack` f m $c = f\ a_0\ b_0\ (f\ a_1\ b_1\ (f \ldots (f\ a_{n-1}\ b_{n-1}\ c) \cdots))$

Example: Airport Luggage System

```
type Lid = string
type Flight = string
type Airport = string
type Route = (Flight * Airport) list
type LuggageCatalogue = Map<Lid,Route>
type ArrivalCatalogue = Map<Airport,Lid Set>
```

Exercises

From the 2013 exam
https://www.imm.dtu.dk/ mire/FSharpBook/OldExams/02157-2013.pdf consider

- Problem 1 (Multisets)

Finish the airport luggage system implementation using Maps.